

XRPL Validator — Daily Engineering Report • 2026-09-03

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **39 commits** (xrpl-validator 39)
- 1 test / vector commit

Engineering changes

- **fix(ledger): single-path bridge legs take and judge the pool offer rippled's tip() hands back (findings 103, 111)** (32512b0 , xrpl-validator) rippled's BookStep runs tryAMM(tip) before the tip's own offer. In single-path mode getOffer(tip) anchors a pool offer at the tip's quality (changeSpotPriceQuality) whenever the pool's spot beats the tip by more than $1e-7$ — or hands back maxOffer when the tip sits beyond the limit — and that offer is consumed FIRST; the tip's own level follows on a later pass (execOffer breaks on the quality change). tip() judges ...
- **fix(ledger): a dust offer whose quality rounds worse is removed, not crossed (finding 112)** (9534ff1 , xrpl-validator) rippled's OfferStream::step (fixRmSmallIncreasedQOffers) drops an offer whose effective amounts — clipped to the owner's funds with ceilOutStrict when it is not the issuer and short — have a TakerPays at the asset's smallest positive unit (one drop) and a quality, recomputed from those amounts, worse than the directory's; TakerGets in XRP is never subject to it, and two IOUs only when TakerPays < TakerGets. The ...
- **fix(ledger): erasing a sender's in-flight line removes its entries, not the pages (finding 113)** (029a608 , xrpl-validator) Intermediate acquisitions ride through the SENDER in our strand walk: a hop credits the sender's line in the hop currency and the next hop debits it, and the net-zero line drops out of the mutation set. When that line did not exist, its creation also inserted the sender's key into two owner directories, and the walk erased those traces by restoring the sender's line, both directory roots and both LAST pages from ...
- **fix(ledger): killed crossings still reap what the rev pass stepped over (finding 114)** (f55ce32 , xrpl-validator) #106730661 3241437EB11D — a tfImmediateOrCancel buy of 20.575959 XRP for 28.215814 RLUSD by a taker holding 0.0000206 RLUSD. Mainnet: tecKILLED, nothing crossed, and the two EXPIRED offers 729F17CE / 392EFC07 on the level behind the tip are deleted with their page. Ours: tecKILLED, nothing deleted (differential 84/85).
- **fix(ledger): the rev pass steps past a maker it emptied and reaps behind it (finding 115)** (f7156c9 , xrpl-validator) #106731793 E3E8726FA355 — rGodbj1g sells 1.5 XRP for XRPCAT (tfSell|tfImmediateOrCancel). Mainnet: 653713 XRPCAT taken from rFD9A7M8's 01EA78BD, and rn2zm7j6's unfunded 1F26E638 AND 8BB6A1F0 — on two different levels, the second behind rFD9A7M8's live 9EABC3F5 — deleted. Ours (F114): 1F26E638 reaped, 8BB6A1F0 kept; the live shadow then carried it through the ledger — present-vs-deleted, the book page 32 bytes long, ...

- **fix(ledger): judge the taker's limit by the level's quality, not the offer's own rate (finding 116)** (67140df , xrpl-validator) #106732842 7BD990E595DF — rKv64MDJ buys 19.718743 "666" for 5.783103 XRP. Mainnet crosses rJ4EpEPT's underfunded 16DAB624 for its whole 12.709 "666" (3727433 drops), deletes the offer and its owner page, and rests the taker's 7.00924955970754 remainder. Ours crossed nothing and rested the full offer: six maker-side nodes missing from our write-set, the taker's root and resting offer off.
- **fix(ledger): a level this apply emptied is stepped through, not reported live (finding 117)** (b0e848e , xrpl-validator) #106733197 DFD1E132035C — rpK3DLr8 buys 3111.618064 XRP for RLUSD (tfImmediateOrCancel). Mainnet takes three underfunded bot makers whole on three levels (0.8, 0.6, 0.4 XRP) and deletes six more of their offers with their pages: the dead ones behind each fill and rD7q6194Bw's remaining D8118C65 and 119270E3, two levels further on. Ours took the same three fills and left those two behind — the maker's OwnerCount one ...
- **fix(ledger): the first book hop composes its in-side transfer rate into the strand's quality function (finding 119)** (44348d2 , xrpl-validator) #106732759 34694521561A — a self-payment of 16.45 UNI for SendMax 100.32264774 USDT, tfPartialPayment|tfLimitQuality, six paths, two of them through AMM pools (USDT/XRPS, XRPS/UNI). Mainnet delivers 15.62 UNI; ours 5.83 — six lines off, the two pools' balances among them.
- **fix(ledger): the remaining-in budget is an STAmount (finding 118)** (f852b6b , xrpl-validator) #106733048 E4014705 — a tfSell of 8267788.778091 CSC for LHT: 109292.827 CSC at the first level, the rest at a 1:1 maker. Mainnet credits the taker 8267867.899868732 LHT; ours 8267867.899868731. #106732165 185A4B50 — a tfSell|tfIoC of 200 EVR under a 1.002 issuer rate: the maker's residual TakerPays is 332.7392751903707 on mainnet, 332.7392751903708 in ours.
- **fix(ledger): the pool turn anchors on the residual tip inside the inflated limit — the taker's own tip included (findings 120, 121)** (e9f2d58 , xrpl-validator) Two faces of one rippled rule.
tryAMM(offers.tip().quality()) anchors the pool's offer on the RAW head of the book, and BookOfferCrossingStep::qualityThreshold(lobQuality) unanchors it (the limitOut-sized max offer) only when qualityThreshold_ > lobQuality, i.e. the tip is WORSE than the transfer-rate-inflated limit (BookStep.cpp). The head is whatever the stream lands on — the taker's own resting offer ...
- **fix(ledger): the funds-exhausting fill takes the line's own balance; a bridged round's multiPath and self-tip removal follow rippled's pass (findings 122, 123; finding 121 refined)** (4199029 , xrpl-validator) Finding 122 — #106734972 E7CB46461F09 and #106734622 C1183E11AEC3, two IoC sales that run the taker's line dry. rippled's last, funds-limited fill takes what the LINE still holds — the DirectStep's maxSrcToDst, the balance as the ledger carries it after every earlier iteration's 16-digit half-even debit — 653.9861023930549 after thirty-eight debits where the budget's sorted-fold remainder (finding 118) says ...550, ...
- **fix(ledger): the out-limited input is clamped to the underfunded maker's own input (finding 124)** (6abd978 , xrpl-validator) #106735594 982E2CDBD610 — rapido pays 605278 drops for 2.065196669826816 "666" through rJ4EpEPT's offer, funded for 2.06519666982682 of its 20.021509. Mainnet takes 605277 drops; ours took 605278 — the maker's root and residual offer one drop off.
- **fix(ledger): a bridged round ranks the direct strand on the book's live tip, not the emptied level under the cursor (finding 125)** (fdb2592 , xrpl-validator) rippled's activateNext sorts the strands on qualityUpperBound — tipOfferQuality, a BookTip::step over the view as it stands when the iteration opens: the winning strand's sandbox and every stream's dead offers were applied at the END of the previous iteration (StrandFlow.h, best->sb.apply(sb) and the ofrsToRm offerDelete loop), and BookTip reads the directory, not the offer, so a self-offer or an ...

- **fix(ledger): the mixed engine's book segment nets its carry at mulRatio-nearest (finding 126)** (b6f8c45 , xrpl-validator) The fifth floor of the net-division family. rippled's `limitStepIn` nets the input a book step is fed at `mulRatio(stpAmt.in, QUALITY_ONE, trIn, false)` — IOUAmount's `mulRatio`: the quotient carried to eighteen digits, normalized to sixteen at Number's half-even nearest, and only then bumped by the `roundUp` flag. `strand_pass`'s hop joint, `spend0`, the live-line clamp and the hop `avail` all moved to `mul_ratio` for ...
- **fix(ledger): an XRP side of a maker's residual subtracts in exact drops (finding 127)** (e7b7927 , xrpl-validator) `offer_residual` rounded every maker remainder back to sixteen significant digits — `Toffer::consume` subtracts through `STAmount`, and for an IOU that is Number's half-even nearest. For a native side it is not: `STAmount` holds drops in an `int64` and `operator-` is exact.
- **fix(ledger): a bridged strand's limit-out function composes the tip's offer, not the forced pool (finding 128)** (8dca547 , xrpl-validator) rippled's `limitOut` builds the strand's quality function step by step from `getQualityFunc` → `tipOfferQualityF` → `tip()`, and `tip()` hands back the pool's offer only when the offer it would GENERATE beats the book's tip on its own quality (`ammOffer->quality() > lobQuality`, `BookStep.cpp`). The single-path forcing — `qualityThreshold` returning `nullopt` when the strand's limit reads better than the leg's tip, a ...
- **fix(ledger): an encoded rate rounds to nearest after the legacy half-up digit; the fib seed multiplies at nearest (findings 129, 130)** (66db675 , xrpl-validator) Two rounding rules in `amm_swap.rs`, each with its own vectors, committed together because they share the file and the same drill set proved them.
- **fix(ledger): a DepositPreauth entry carries Flags and its owner-directory node (finding 132)** (fc91ef2 , xrpl-validator) rippled's `DepositPreauth::doApply` builds the entry, inserts its key into the owner directory and records the page in `sfOwnerNode`; every ledger entry carries `sfFlags`. The transactor here was the early simplified one: it wrote the object bare — no `Flags`, no `OwnerNode` — and never touched the directory, and it answered an existing authorization with success where rippled's `preclaim` says `tecDUPLICATE` (and ...
- **fix(ledger): a forward-driven pool turn swaps the carried input and clamps only the delivery (finding 131)** (ad7b3eb , xrpl-validator) rippled sizes a strand backwards and, once a step is limited — `SendMax` at the source, or a step that cannot supply its request — resets the sandbox and runs every later step forwards, IN-driven: `BookStep::fwdImp` consumes offers by input, and a single-path pool re-swaps the carried input (`limitStepIn` → `AMMOffer::limitIn` → `swapAssetIn`). The reverse cache reinstates the reverse amounts only when ...
- **fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)** (52b34c3 , xrpl-validator) `BookStep::fwdImp`'s reconciliation: when the forward output exceeds the reverse cache, `limitStepOut(cache.out)` re-sizes the offer to the reverse amounts, and if THAT step input — the reverse net re-grossed by the in-side transfer rate — equals the forward input as carried, the reverse amounts stand and the whole gross is consumed; the fee absorbs the sliver. Finding 131 swept every tiny sliver into a single ...
- **fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)** (c8774b1 , xrpl-validator) Finding 137 — rippled's `BookStep::forEachOffer` consumes, in ONE pass, every offer whose quality equals the first offer's (`else if (*ofrQ != offer.quality()) return false;`) — the whole same-quality directory group — and the flow iterates only when the quality changes. Ours took one leg-B offer per iteration. The pool on leg A advances its Fibonacci slice per ITERATION (`AMMContext::curIters`), so a ladder of ...
- **fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)** (f6c5d75 , xrpl-validator) rippled's `replaceSignerList` (`SignerListSet.cpp:299-355`) is remove-then-create, for a fresh list and a replacement alike: `removeSignersFromLedger` = `dirRemove` at the old list's `OwnerNode` hint + `OwnerCount` -1

(`IsOneOwnerCount`; a legacy list without the flag gives back $2 + n$) + erase; then

`checkReserve(preFeeBalance_, ownerCountDelta = 1)`; then a fresh SLE, `dirInsert` — the key lands at the END of the owner ...

- **fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)** (e7b4239, xrpl-validator) `rippled OfferCreate.cpp:835-851` runs the reserve check after crossing, unconditionally: `preFeeBalance_` (the creator's balance before this transaction's fee, `Transactor.cpp:896`) against the reserve for one more object at the CURRENT owner count. Below it the remainder is never placed — `tecINSUF_RESERVE_OFFER` when nothing crossed, `tesSUCCESS` with the fills kept and the residual dropped when something did ("If ...
- **fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)** (b28172a, xrpl-validator) Two faces of `rippled's PaymentSandbox`, committed together because they share `offer.rs` and one drill set proved them.
- **fix(ledger): strands are admitted against the fee-inclusive limit; self-offers leave the book only when the direct strand executes (finding 139)** (168f252, xrpl-validator) `rippled` has ONE `limitQuality` for the crossing — `Quality{takerAmount.out, sendMax}` with the gateway transfer fee INSIDE `sendMax` (`OfferCreate.cpp: 380-393`) — and holds the strands' RAW upper bounds against it (`adjustQualityWithFees` returns the tip's own quality for CLOB and multi-path pool offers): strand activation (`StrandFlow.h activateNext`), the per-strand `qualityUpperBound < limitQuality` skip, and the ...
- **fix(ledger): a path that only names the SendMax issuer at its head is the default strand (finding 140)** (2399f83, xrpl-validator) `rippled's toStrands` inserts a strand only if it is not already in the list (`hasStrand`, `PaySteps.cpp`). A named path [`rMxCK` (the `SendMax` issuer), `XRP`] normalises to exactly the default strand [`src → issuer, RLUSD/XRP book`] — the head hop `src → SendMax issuer` is on every strand — so the two collapse to ONE strand and the flow runs single-path.
- **fix(ledger): an unanchored pool tips its leg by SPOT, and the single pass is sized to the limit (finding 141)** (a4080bb, xrpl-validator) `AMMLiquidity::maxOffer` — the offer a single-path leg gets when its book tip is worse than the strand's limit (`qualityThreshold` returns `nullopt`, no anchor) — carries `Quality{balances}`, the pool's feeless SPOT, as its quality: `AMMOffer(*this, {swapAssetOut(balances, out), out}, balances, Quality{balances})`. The tip compare that follows, `ammOffer->quality() > lobQuality`, is therefore spot against the book, ...
- **fix(ledger): iteration 0 takes multiPath from the strands activateNext keeps (finding 142)** (03e305a, xrpl-validator) `flow()` (`StrandFlow.h`) runs `activateNext(sb, limitQuality)` at the TOP of every iteration — the first included — and only then `ammContext.setMultiPath(activeStrands.size() > 1)`: a strand whose upper bound misses `limitQuality` is dropped before any pool is asked for an offer. Ours handed round 0 the built-strand entry (`multi_prev, "strands.size() > 1"`), which is what `Flow.cpp` sets before the loop and what the ...
- **fix(ledger): the crossing's residual is the original less the STAmount sum of the iterations' fills (finding 144)** (2155052, xrpl-validator) `flow()` accumulates each iteration's actual in and out in `savedIns / savedOuts`, and `OfferCreate::flowCross` derives the residual from `sum(savedIns)` (`tfSell: afterCross.in -= divideRound(actualIn, xferRate, up)`) or `sum(savedOuts)` (a buy: `afterCross.out -= actualOut`), the counterpart then priced through the original rate (`OfferCreate.cpp:497-520`). The sum canonicalizes at every `+=`; a running ...
- **fix(ledger): the direct pool is consulted in a single-path round only if its strand survived activation (finding 145)** (6e3466f, xrpl-validator) `activateNext` (`StrandFlow.h`) keeps a strand only if its upper bound meets `limitQuality`, and that bound is composed BEFORE the round's `setMultiPath`: with the previous iteration's

multiPath, the direct book's tip is the pool's Fibonacci slice at the current iteration count. Only a kept strand is flowed; a single-path round then asks the pool for its anchored (or limit-sized) offer.

- **fix(ledger): a strand's first pool hop is sized by the reverse pass, not swept to the SendMax (finding 147)** (47905e9 , xrpl-validator) The tiny-silver sweep in `consume` (finding 131) models `BookStep::fwdImp` consuming the WHOLE product a previous step handed it: when the input's unspent remainder is under one part in a billion, the pool takes it all and the overshoot flows on. A strand's FIRST book step is not fed by a product. Its forward input is the reverse pass's own answer (`limitStepOut` → `swapAssetOut(want)`), already the exact in, ...
- **fix(ledger): re-run a limiting book level and re-price its last maker (finding 149)** (e801434 , xrpl-validator) rippled's strand driver treats a book step that could not satisfy the requested output as LIMITING and re-executes it in reverse with the step's own result as the target (`StrandFlow.h:176-184`, `stepOut = r.second`). `BookStep::revImp` then walks the same offers a second time with `remainingOut = out - sum(savedOuts)` after each, and the level's last offer is trimmed by `limitStepOut` to that remainder instead of its ...
- **fix(ledger): record a credential's directory pages (finding 150)** (bafc30c , xrpl-validator) `CredentialCreate` stores the page of each owner directory the credential is inserted into — `sleCred->setFieldU64(sfIssuerNode, *page)` after the issuer's `dirInsert` and `sfSubjectNode` after the subject's, none of the latter on a self-issued credential (`CredentialCreate.cpp doApply`). Both are `UInt64` fields with two-byte headers, ten bytes each; our object carried neither, so every credential we created was 20 bytes ...
- **fix(ledger): a strand that flows nothing never returns (finding 151)** (802b6a7 , xrpl-validator) rippled's payment driver keeps its candidate strands in `ActiveStrands: next_` is seeded with every strand, and each iteration `activateNext` turns it into the sorted active set, sorting and filtering (no bound, or a bound past `limitQuality`) only when more than one strand is pending — a lone pending strand is activated unexamined. The iteration then pushes back only the strand that produced (unless it went ...
- **fix(ledger): an expired check cannot be cashed or created (finding 152)** (02653b5 , xrpl-validator) rippled judges a check's `Expiration` against the parent close time — `hasExpired(view, exp)` is `parentCloseTime >= *exp` — and returns `tecEXPIRED` from `CheckCash::preclaim` (`CheckCash.cpp:132-136`), after the destination check and before any funds move; `CheckCreate::preclaim` applies the same rule to a check created already expired (`CheckCreate.cpp:179-183`). We modelled neither.
- **fix(ledger): the direct pass removes a self-offer only inside the limit, and stops at one beyond it (finding 143)** (399e5b4 , xrpl-validator) `limitSelfCrossQuality` (`BookStep.cpp`) deletes the taker's own offer off the direct tip only when `offer.quality() >= qualityThreshold_`; one beyond the limit falls through to `checkQualityThreshold`, which fails, and the pass ENDS there. Finding 139 moved the reap onto the direct strand's own walk and, in doing so, dropped the quality gate the round-opening reap had carried: `live_head(mutate_self)` deleted every ...
- **fix(ledger): an offer that became unfunded during the crossing is not removed for good (finding 153)** (18ff76e , xrpl-validator) rippled's offer stream deletes every offer it steps past in the iteration's sandbox (`BookTip::step` removes the previous entry), but only an offer that was ALREADY unfunded when the iteration began is added to the permanent-removal set: `OfferStream::step` compares the owner's funds in the cancel view with the funds now and, when they differ, logs "became unfunded" without `permRmOffer` (`OfferStream.cpp:279-304`). The ...
- **fix(ledger): a stranger may burn an NFToken only if it is burnable and they are its issuer or minter (finding 154)** (8405e57 , xrpl-validator) `NFTokenBurn::preclaim` lets a token's owner burn it unconditionally; any other account needs the token flagged burnable (0x0001 in the `NFTokenID`'s flag word) and must be its issuer or the

issuer's current NFTokenMinter — an issuer with no minter set fails that comparison, an issuer whose root is gone skips it (NFTokenBurn.cpp preclaim). We had no permission check at all: every burn with an Owner field went through.

Tests and campaigns

- **test(node): bundles rebuild in-ledger objects exactly and can pin untouched targets** (0bbd06e, xrpl-validator)

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **MPC Threshold-Signature Policy Gate** (custody) Add a new AccountPolicy ledger object (ltACCOUNT_POLICY, owner-reserved, keylet::accountPolicy(account)) set via a PolicySet transaction, carrying
- **Oracle-Triggered Stablecoin Circuit Breaker** (tokens) Add a circuit-breaker config to MPTokenIssuance — sfReferenceOracles (a bounded STVector256 of up to 8 PriceOracle keylets) and sfDepegThresholdBPS
- **Weighted Basket Vaults for Index Funds** (lending) Add a tfBasket flag to VaultCreate and a bounded sfBasketAssets STArray (max 10 {Asset, WeightBPS} entries summing to 10000) stored on the Vault S

Appendix — commits

Time	Repo	Commit	Subject
01:22	xrpl-validator	32512b0	fix(ledger): single-path bridge legs take and judge the pool offer rippled's tip() hands back (findings 103, 111)
02:22	xrpl-validator	9534ff1	fix(ledger): a dust offer whose quality rounds worse is removed, not crossed (finding 112)
05:16	xrpl-validator	029a608	fix(ledger): erasing a sender's in-flight line removes its entries, not the pages (finding 113)
06:33	xrpl-validator	f55ce32	fix(ledger): killed crossings still reap what the rev pass stepped over (finding 114)
07:26	xrpl-validator	f7156c9	fix(ledger): the rev pass steps past a maker it emptied and reaps behind it (finding 115)
08:05	xrpl-validator	67140df	fix(ledger): judge the taker's limit by the level's quality, not the offer's own rate (finding 116)
08:15	xrpl-validator	b0e848e	fix(ledger): a level this apply emptied is stepped through, not reported live (finding 117)
09:13	xrpl-validator	44348d2	fix(ledger): the first book hop composes its in-side transfer rate into the strand's quality function (finding 119)
09:13	xrpl-validator	f852b6b	fix(ledger): the remaining-in budget is an STAmount (finding 118)

Time	Repo	Commit	Subject
10:00	xrpl-validator	e9f2d58	fix(ledger): the pool turn anchors on the residual tip inside the inflated limit — the taker's own tip included (findings 120, 121)
10:45	xrpl-validator	4199029	fix(ledger): the funds-exhausting fill takes the line's own balance; a bridged round's multiPath and self-tip removal follow rippled's pass (findings 122, 123; finding 121 refined)
10:53	xrpl-validator	6abd978	fix(ledger): the out-limited input is clamped to the underfunded maker's own input (finding 124)
11:37	xrpl-validator	fdb2592	fix(ledger): a bridged round ranks the direct strand on the book's live tip, not the emptied level under the cursor (finding 125)
11:37	xrpl-validator	b6f8c45	fix(ledger): the mixed engine's book segment nets its carry at mulRatio-nearest (finding 126)
12:02	xrpl-validator	e7b7927	fix(ledger): an XRP side of a maker's residual subtracts in exact drops (finding 127)
12:15	xrpl-validator	8dca547	fix(ledger): a bridged strand's limit-out function composes the tip's offer, not the forced pool (finding 128)
12:25	xrpl-validator	66db675	fix(ledger): an encoded rate rounds to nearest after the legacy half-up digit; the fib seed multiplies at nearest (findings 129, 130)
13:18	xrpl-validator	fc91ef2	fix(ledger): a DepositPreauth entry carries Flags and its owner-directory node (finding 132)
13:18	xrpl-validator	ad7b3eb	fix(ledger): a forward-driven pool turn swaps the carried input and clamps only the delivery (finding 131)
15:33	xrpl-validator	52b34c3	fix(ledger): a forward-swept pool turn reconciles to the reverse amounts when the re-grossed reverse input equals the carried gross (finding 131, refined)
15:48	xrpl-validator	c8774b1	fix(ledger): a book pass takes the whole same-quality group; pool slices price by their 16-digit rate (findings 137, 138)
15:48	xrpl-validator	f6c5d75	fix(ledger): SignerListSet replace re-inserts the owner-directory entry (finding 136)
15:48	xrpl-validator	e7b4239	fix(ledger): reserve gate applies to crossed offers on the pre-fee balance (finding 135)
15:48	xrpl-validator	b28172a	fix(ledger): the taker's spend is capped by the deferred-credits balance; a maker's XRP reserve holds at the flow's original owner count (findings 133, 134)
15:54	xrpl-validator	168f252	fix(ledger): strands are admitted against the fee-inclusive limit; self-offers leave the book only when the direct strand executes (finding 139)
15:56	xrpl-validator	2399f83	fix(ledger): a path that only names the SendMax issuer at its head is the default strand (finding 140)
16:09	xrpl-validator	a4080bb	fix(ledger): an unanchored pool tips its leg by SPOT, and the single pass is sized to the limit (finding 141)
17:35	xrpl-validator	03e305a	fix(ledger): iteration 0 takes multiPath from the strands activateNext keeps (finding 142)
18:12	xrpl-validator	2155052	fix(ledger): the crossing's residual is the original less the STAmount sum of the iterations' fills (finding 144)
18:12	xrpl-validator	6e3466f	fix(ledger): the direct pool is consulted in a single-path round only if its strand survived activation (finding 145)
18:47	xrpl-validator	47905e9	fix(ledger): a strand's first pool hop is sized by the reverse pass, not swept to the SendMax (finding 147)
20:16	xrpl-validator	e801434	fix(ledger): re-run a limiting book level and re-price its last maker (finding 149)
20:21	xrpl-validator	bafc30c	fix(ledger): record a credential's directory pages (finding 150)
20:42	xrpl-validator	802b6a7	fix(ledger): a strand that flows nothing never returns (finding 151)

Time	Repo	Commit	Subject
20:44	xrpl-validator	02653b5	fix(ledger): an expired check cannot be cashed or created (finding 152)
22:26	xrpl-validator	399e5b4	fix(ledger): the direct pass removes a self-offer only inside the limit, and stops at one beyond it (finding 143)
22:41	xrpl-validator	0bbd06e	test(node): bundles rebuild in-ledger objects exactly and can pin untouched targets
23:33	xrpl-validator	18ff76e	fix(ledger): an offer that became unfunded during the crossing is not removed for good (finding 153)
23:54	xrpl-validator	8405e57	fix(ledger): a stranger may burn an NFToken only if it is burnable and they are its issuer or minter (finding 154)